Part 1b: Getting started in R and Posit

Quantitative Session of the 2026 HUSOE ELPS Methods and Data Institute

Nathan Alexander, PhD

Table of contents

0.1 Using Posit Cloud

Posit Cloud is a web-based platform that allows users to perform data science tasks directly in their browser. It provides a cloud-based environment similar to the traditional RStudio integrated development environment (IDE), eliminating the need for local software installation and maintenance. Users can create projects, share work with collaborators, and access features like interactive notebooks. Posit Cloud offers various plans, including a free option for casual use and premium plans for professionals, instructors, and organizations.

You can access the Posit Cloud here.

0.2 Layout

When you get into Posit Cloud you will see the following layout:

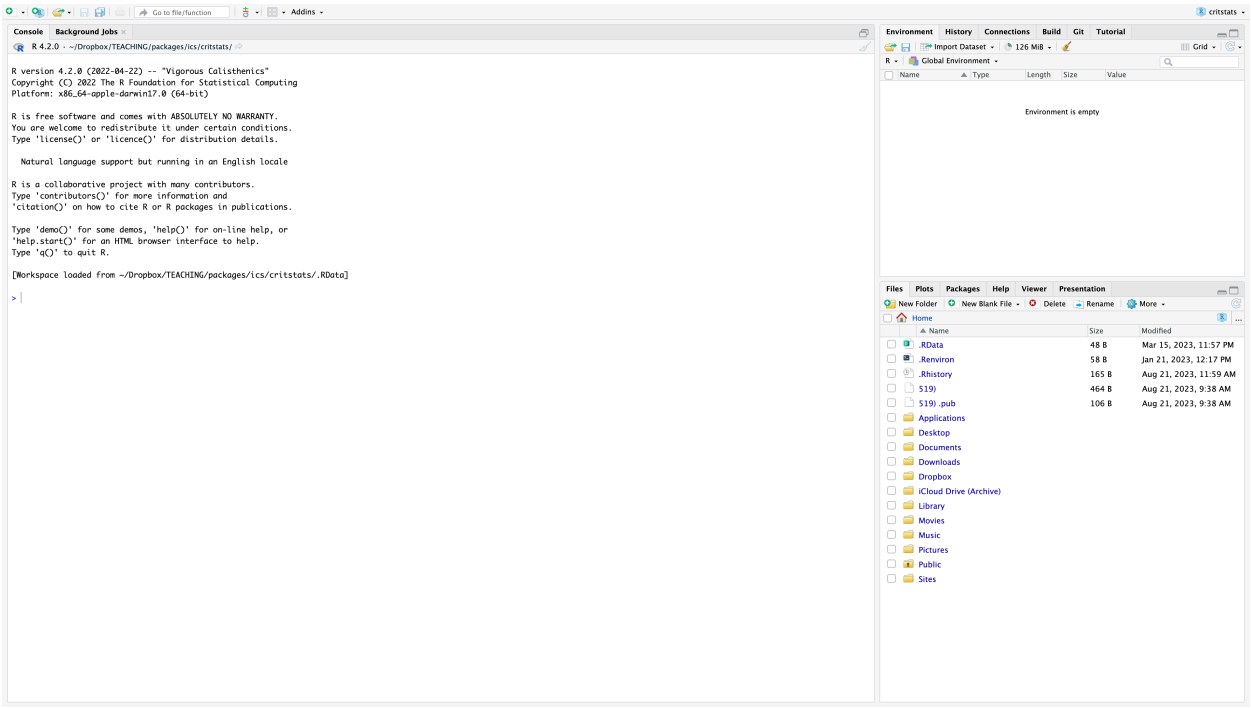

Figure 1: RStudio Panel

This panel include the Console on the left and the options tabs on the right.

When we open up our first file, a Markdown file, you will see the panel change as follows.

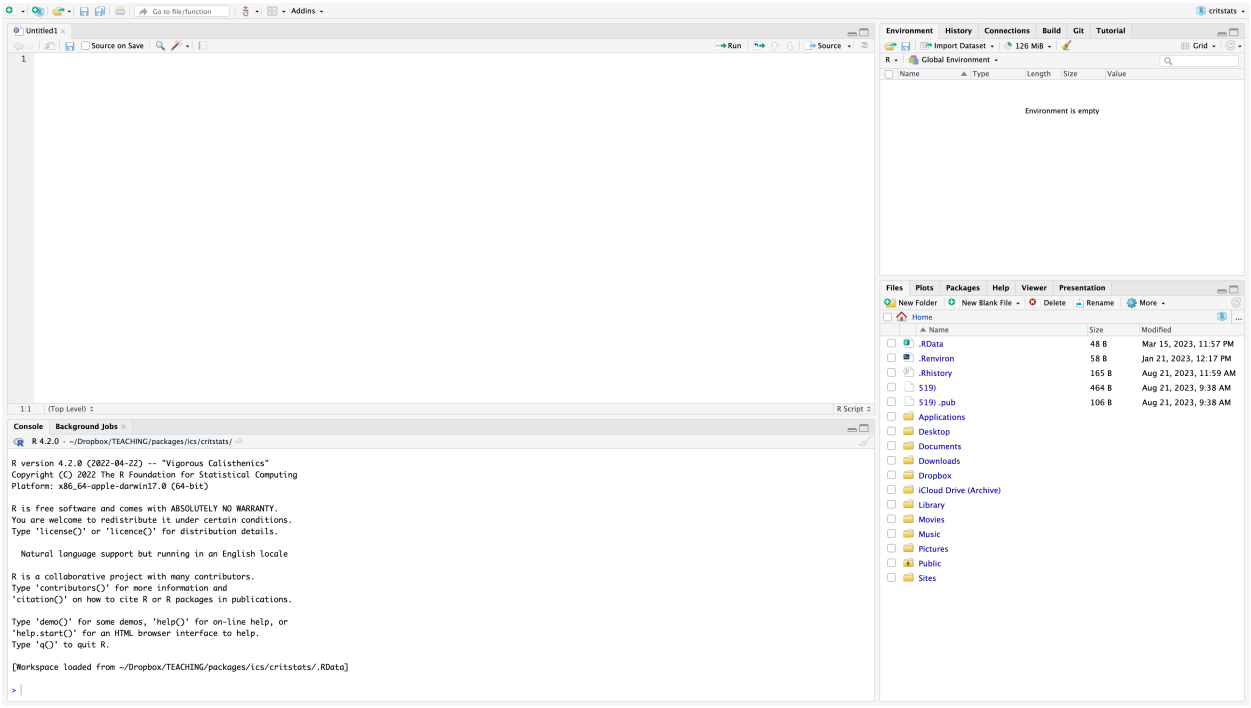

Figure 2: RStudio Panel

We will work from the top left panel, in the Markdown file, and see our output in the Console on the bottom left.

0.3 Directories

Directories help us organizer our R projects efficiently.

A common structure includes a root directory that helps set the working directory automatically. Within this root, subdirectories such as "Data" for storing datasets, "src" or "R" for R scripts and functions, "output" for results, and "plots" for visualizations are typically used. This organization enhances project management, improves collaboration, and simplifies file referencing. Using relative file paths allows for more portable code, making it easier to share projects across different computers or with collaborators.

0.3.1 See your current working directory

To see your current working directory, we'll run the `getwd()` command.

```
getwd()
```

0.3.2 List files in your working directory

To view files in your working directory, we'll run the `list.files()` command.

```
list.files()
```

0.4 Packages

Packages in R are collections of functions, data, and documentation that extend the language's capabilities. They allow users to easily share and reuse code, making R a powerful and flexible tool for data analysis and visualization.

Packages can be installed from repositories like CRAN, and once installed, they can be loaded into an R session to access their functionality. The tidyverse, for example, is a popular collection of packages that provides a consistent and user-friendly approach to data manipulation and visualization.

0.4.1 Install the Praise package.

```
# install the package
install.packages("praise", repos = "http://cran.us.r-project.org")
```

```
The downloaded binary packages are in
    /var/folders/t2/v2ypghs120z45678ty2nqfk00000gn/T//Rtmpb7rGI7/downloaded_packages
```

0.4.2 Load the Praise package

```
# load library
library(praise)
```

0.4.3 Now get some Praise!

```
# get some praise
praise()
```

```
[1] "You are praiseworthy!"
```

0.5 Arithmetic in R

We will learn how to calculate values in R.

```
1 + 2  # the 'plus sign' computes the sum
```

```
[1] 3
```

```
2 - 3  # the 'minus sign' computes the difference
```

```
[1] -1
```

```
3 * 4  # the 'asterisk' computes the product
```

```
[1] 12
```

```
4 / 5 # the 'forward slash' computes the quotient
```

```
[1] 0.8
```

```r
# we can also compute the sum of the first 100 positive integers
sum(1:100)
```

```
[1] 5050
```

Explore different object types

For this task, we will explore three object types: *numeric*, *character*, and *logic* values.

0.5.0.1 Task 2-a: Compute a mathematical statement and create a numeric variable

```r
1 + 2
```

```
[1] 3
```

We can assign a variable to this statement by using an assignment operator: <-

```r
a <- 1 + 2
```

We can also use an equal sign to assign values: =

```r
a = 1 + 2
```

Type "a" to show the value of the variable

```r
a
```

```
[1] 3
```

Create a numeric variable "b" that is the product of "a" and "5"

```r
b = a*5
```

Type "b" in your console to show the product of the two variables

```r
b
```

```
[1] 15
```

Divide b by 4

```r
b / 4
```

```
[1] 3.75
```

Take the square root of b

```r
sqrt(b)
```

```
[1] 3.872983
```

Compute the natural log of b

```r
log(b)
```

```
[1] 2.70805
```

Compute the common log of b

```r
log10(b)
```

```
[1] 1.176091
```

Find 1 minus the square root of b

```r
1-sqrt(b)
```

```
[1] -2.872983
```

Attempt to find the square root of "1 minus the square root of b" - which is a negative value

```r
sqrt(1 - sqrt(b))
```

```
Warning in sqrt(1 - sqrt(b)): NaNs produced
```

```
[1] NaN
```

NaN stands for "Not a number". This occurs because there is currently no defined value to recognize the square root of negative numbers in R. But we can compute the square root on the absolute value of this difference, if needed.

```
sqrt(abs(1-b))
```

```
[1] 3.741657
```

We can insert longer or more complex mathematical statements too. For example, we can find the absolute value of the sum of -1 and the square root of b cubed and then subtract from that the value of 3 times the square root of b.

Notice the use of parentheses.

```
abs(-1+sqrt(b^3)) - 3*(sqrt(b))
```

```
[1] 45.4758
```

We can override the original value of y to match the mathematical statement we generated above.

```
y <- abs(-1+sqrt(b^3)) - 3*(sqrt(b))
y
```

```
[1] 45.4758
```

We consider all of the previous objects to be numeric.

We can also create objects to hold non-numeric values.

There are two types of non-numeric values: *character* values and *logic* values.

0.5.0.1.1 Character values

```
character <- 'some label'
character
```

```
[1] "some label"
```

We can create a character value using single (') or double (") quotation marks.

```
character <- "some label"
character
```

```
[1] "some label"
```

0.5.0.1.2 Logic values

Logic values can either be TRUE or FALSE

```
logic_true <- TRUE
logic_false <- FALSE
```

```
logic_true
```

```
[1] TRUE
```

```
logic_false
```

```
[1] FALSE
```

We can also use T for TRUE and F for FALSE.

```
logic_true <- T
logic_false <- F
```

```
logic_true
```

```
[1] TRUE
```

```
logic_false
```

```
[1] FALSE
```

0.6 Variables in R

We will learn to give a variable (or character) a value.

Use the different assignment operators

```
y = 2 # the equal sign can be used as an assignment operator

y <-2 # a "less than" sign and dash can also be used as an assignment operator

y # R stores all values you assign, so you must "call" any variables to see their values
```

```
[1] 2
```

Set x equal to two added to three

```
x = 2 + 3
x
```

```
[1] 5
```

Set y equal to two minus three

```
y = 2 - 3
y
```

```
[1] -1
```

Set z equal to two times three

```
z = 2 * 3
z
```

```
[1] 6
```

Overwrite the value of y by setting y equal to x divided by z

```
y = x / z
y
```

```
[1] 0.8333333
```

0.7 Recommended readings

Baumer, B. S., Kaplan, D. T., & Horton, N. J. (2017). *Modern data science with R*. Chapman and Hall/CRC.

Grolemund, G., & Wickham, H. (2017). *R for data science: Import, tidy, transform, visualize, and model data*. O'Reilly Media. https://r4ds.had.co.nz/

Peng, R. D. (2016). *R programming for data science*. Leanpub. https://bookdown.org/rdpeng/rprogdatascience/

RStudio Education (n.d.). Learning R for Beginners. Online. https://education.rstudio.com/learn/beginner/

Wickham, H. (2019). *Advanced R* (2nd ed.). Chapman and Hall/CRC. https://adv-r.hadley.nz/